CMPE 160 Assignment 2
Super Mario Bros.

Contact:
Alperen Izbudak (muttalip.izbudak@std.bogazici.edu.tr)
Volkan Bora Seki (volkan.seki@std.bogazici.edu.tr)

Due: March 30th, 05:00

Assignment Overview

This assignment presents an object-oriented approach to designing and implementing a 2D platform game inspired by the classic game Super Mario Bros. It offers a nostalgic gameplay experience in which the player progresses through a sequence of levels by overcoming obstacles, interacting with enemies, using portals, and collecting coins along the way. The game must be developed using **Java** and the **StdDraw** graphics library.

Your implementation must integrate platform physics, collision detection, and dynamic enemy interactions. Furthermore, the main objectives of the game include portal teleportation, a level progression system, and logic for collectible objects that track player achievements. A scene from the game is shown in Figure 1 below. You can watch a gameplay video from here.

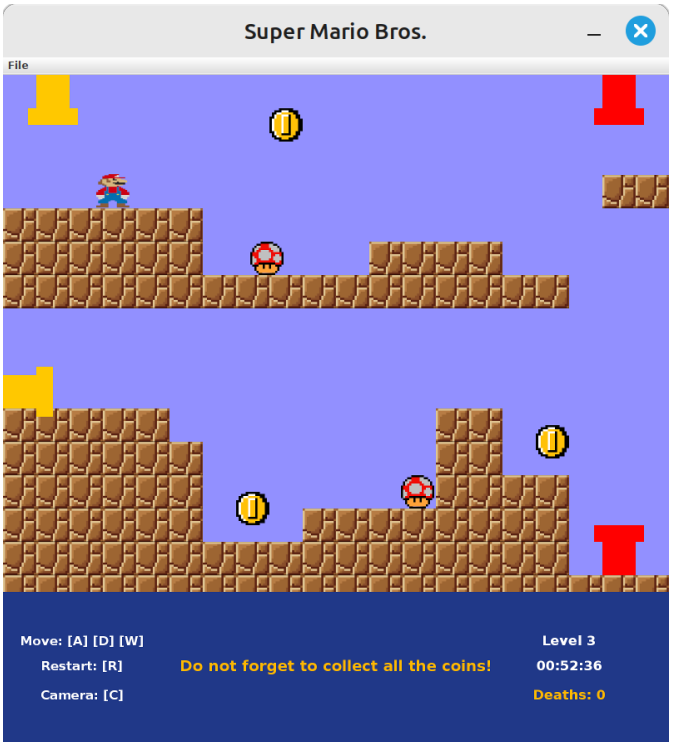


Figure 1: Gameplay overview.

The project must be organized under the `src` directory and use the following core classes: `Main`, `Game`, `Map`, `Mario`, `Enemy` and `Level`. Map coordinates are stored in plain-text files under `data/` and loaded at runtime.

Important: The report **must** be written in LaTeX. Submit both `.tex` and `.pdf`.

Game Assets

This section defines the fundamental game entities and environmental objects, detailing their behaviors and physical properties. All game entities can be seen together in Figure 2. The game consists of 6 entities in total.

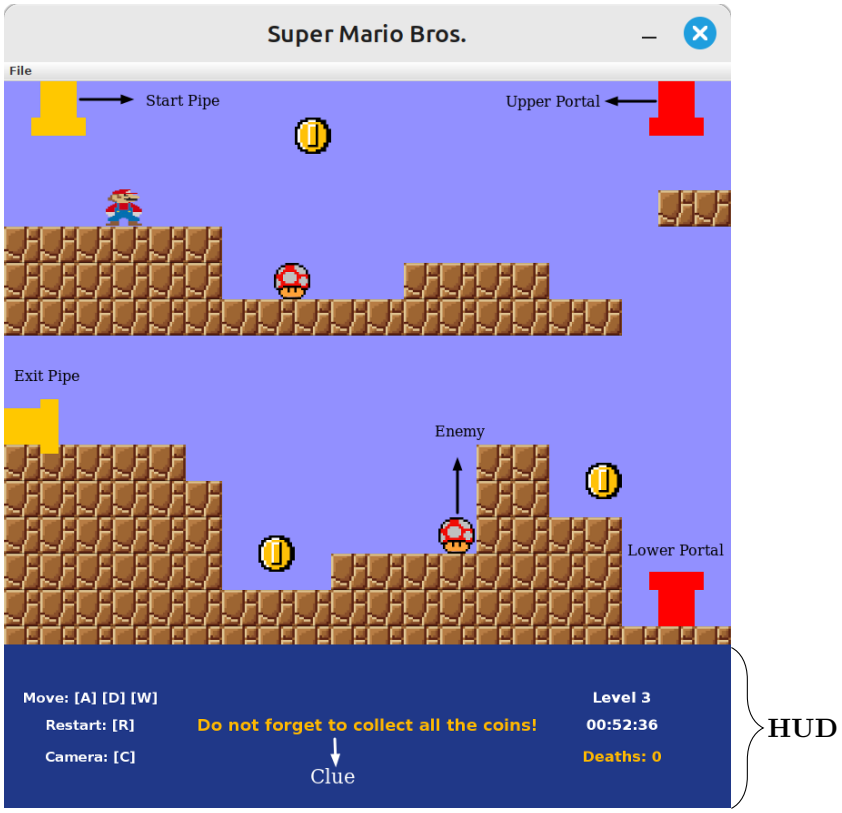


Figure 2: Game assets.

1) Mario

`Mario` is the primary dynamic entity controlled by the user through keyboard inputs. At the beginning of each level Mario falls from the starting pipe and the player can move the Mario to left or right with keys A, D or can jump with W.

Since the player is subject to a vertical gravity constantly, after reaching its maximum height, it should fall with increasing speed. Furthermore if the player contacts with an enemy, its death counter increases and teleport back to the starting coordinates. The `.png` files for depicting each movement of the Mario is provided along assignment files. The `standRight.png` can be seen in Figure 3



Figure 3: Super Mario.

2) Block (Obstacle)

`Blocks` constitute the static architectural layout of the game map. They serve as solid boundaries that nullify the player's velocity vectors, preventing movement through walls or floors.



Figure 4: Obstacle.

3) Coins

`Coins` are provided as the collectible items in the game. When the player touches a coin, it should disappear. If the player dies or resets the current level, the coins must **not** reset.



Figure 5: Coin.

4) Enemy

The `Enemy` is an automated hazard, typically represented as a mushroom sprite, that provides a constant threat to the player. Enemies follow a horizontal path, oscillating between two predefined x-coordinates (x_1 and x_2). If the player touches an enemy bounding box, the player dies, the death counter increases and the player respawns.



Figure 6: Enemy.

5) Portals

`Portals` are spatial transition points that allow the player to move between distant map areas and represented with red rectangles. They function in pairs; entering one designated red zone updates the player's position to the coordinates of the corresponding exit zone.



Figure 7: Portal.

6) Pipes

`Pipes` represents the beginning and end points of each level. They are represented by yellow rectangles. The player respawns at the starting pipe at the beginning of each level, and if the player reaches the exit pipe, they pass the level.

Game Stages

1) Main Menu

The first screen is the main menu, which is used to perform the necessary calibration before the game starts. This screen is provided in Figure 8. Since the game will be executed on computers with different performance characteristics, the menu provides a simple way to adjust the gameplay pace and visual smoothness before entering the actual stages.

Mario can be moved within the menu screen, as long as he remains inside the screen boundaries. This allows the player to observe the current responsiveness of the game during calibration. Pressing **SPACE** starts the game.

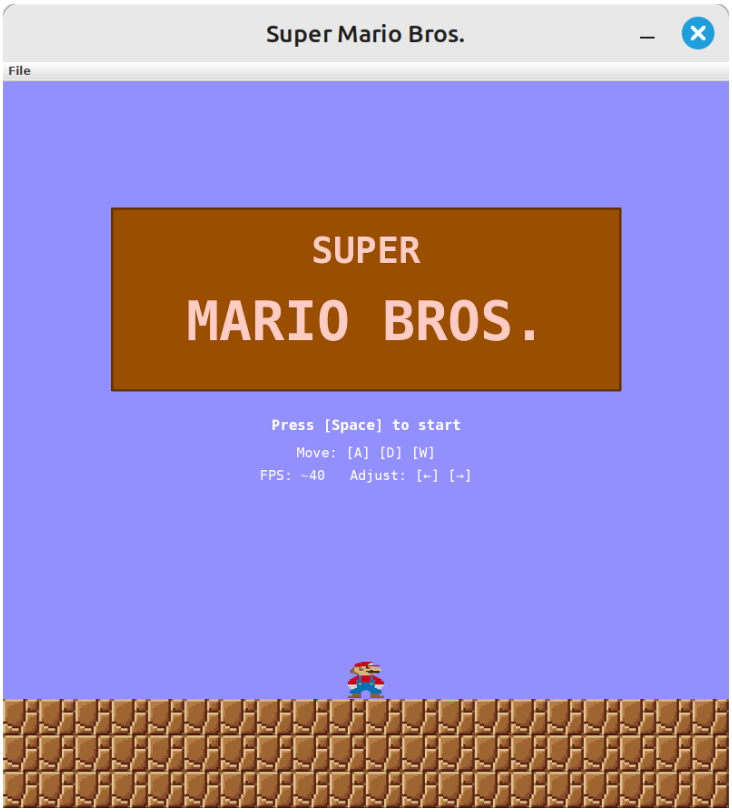


Figure 8: Main menu and calibration screen.

The FPS is approximated from the pause duration used in each iteration of the game loop as

$$\text{FPS} \approx \frac{1000}{\texttt{pauseDuration}}$$

Pressing ⟵ increases the `pauseDuration`, which decreases the FPS. Pressing ⟶ decreases the `pauseDuration`, which increases the FPS. Because the game may run differently across systems, this calibration step is included to reduce visual issues such as flickering and to make the gameplay feel closer to the intended behavior.

2) Game Mechanics

The second stage is the game itself. The player must complete each level without touching any enemy and by fulfilling the given task. Although all levels must include the required game mechanics, their missions and layouts are different.

- **Level 1**: The level should not contain any enemies or coins. The player starts from the starting point, and the level ends when they reach the exit pipe.
- **Level 2**: The level contains two enemies patrolling within given boundaries. The player must complete the level without touching them.
- **Level 3**: There are coins on the map, and the player must collect all of them to complete the level. If the player misses even one coin, they cannot complete the level upon reaching the exit pipe.
- **Level 4**: A custom level designed by the student.

All levels must adhere to the described game mechanics.

Player Movement

- **Movement:** The player uses **A** to move left and **D** to move right.
- **Jumping:** Jumping is triggered by the edge of a **W** key press, not by holding the key down. The player can jump only when on the ground, so double-jumping in mid-air is not allowed.
- **Sprites and Animation:** Several `.png` files are provided for different movement states. The player sprite updates dynamically based on the movement state: idle when standing still, walking while moving on the ground, and jumping while airborne. For walking animation, switching between walk sprites every 8 frames is preferred for smooth and readable motion, although other frame intervals may also be used. For example, when the player walks to the right, the `walkRight1.png` and `walkRight2.png` images should alternate. During jumps, if the player changes direction in mid-air, their facing direction should also change.
- **Death:** After colliding with an enemy, the player switches to the death sprite and a bounce animation plays. The animation ends only after the player falls below the canvas, then the respawn process begins. For exact timing and behavior, watch the gameplay video.

Teleportation

`Portals` are spatial transition points that allow the player to move between distant map areas instantaneously. The portals should behave like obstacles if the player hits them from the sides.

- **Logic Control:** Edge-detection flags must be used so that, after teleportation, the player leaves a portals bounds before another teleportation can occur. This prevents repeated back-and-forth movement between portals.
- **Speed Boost :** After teleportation, the player receives an extra upward velocity boost to safely clear the nearby obstacle. This is an intentional mechanic to keep portal exits smooth and fair.
- **Teleportation Direction:** If Mario uses the upper portal, he is directly teleported to the lower portal. The lower portal, however, behaves like a solid obstacle under normal conditions, so Mario collides with it. To enter the lower portal, Mario must crouch by pressing **S**.

Enemies and Damage

Enemies must patrol between given two coordinates `x1` and `x2`. When the player overlaps with an enemy, play the death animation, then reset the player to the spawn point. The a screenshot depicting the death animation is shown in Figure 9. Upon the player's death, it should respawn at the starting pipe and gloabal death counter displayed ad HUD must be increased.

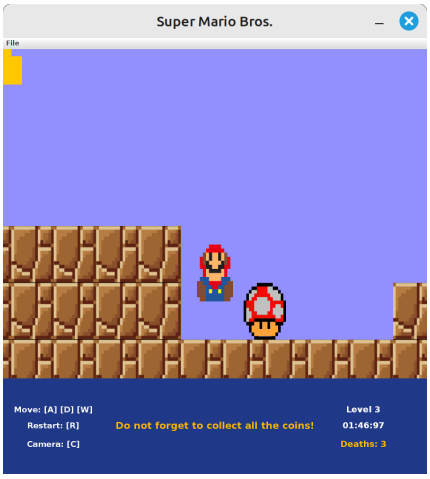


Figure 9: Death animation.

Camera Mode

Pressing C toggles the camera mode.

- **Fullscreen Camera Mode:** This is the initial mode. The camera does not move with the player and shows the entire map/screen.
- **Player Following Camera Mode:** A zoomed-in camera mode that follows the player, keeping the player near the center of the view. The camera tracks player movement, but clamps at the map boundaries (left, right, top, bottom) and remains fixed when a boundary is reached. Zoom is implemented using scale settings. No new obstacles are defined for this mode.

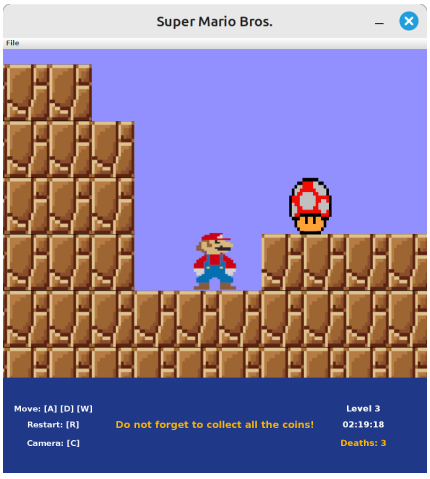


Figure 10: Player following camera mode.

HUD and Time

Display the following information on screen:

- Elapsed time in `MM:SS:ms` format
- Current level
- Death count
- Current level clue text
- Key reminders for controls

Level Completion

Each levels ends when the player reaches to the exit pipe without touching enemies and completing the given task. After completing a level a level-over banner is shown between levels to indicate level progression. User must press **SPACE** to progress. The banner is shown in Figure 11.

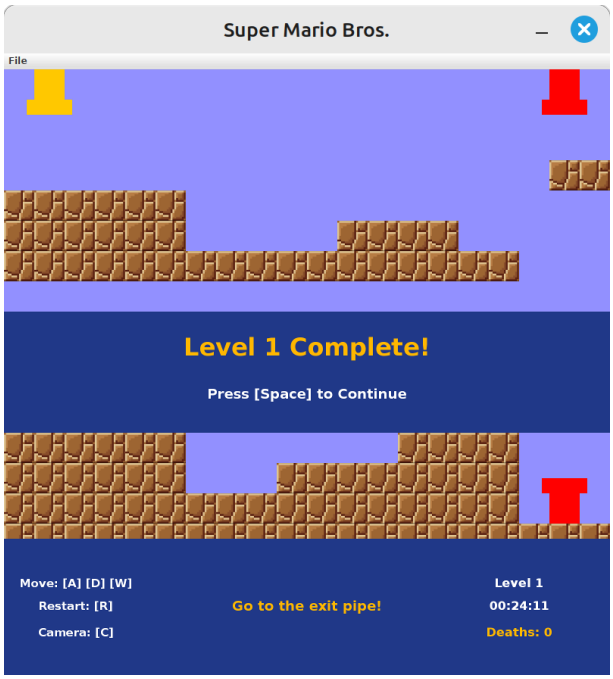


Figure 11: Level completion banner.

Restart

Pressing **R** restarts the game. The player respawns at the starting pipe, returns to level 1, and the game state is reset.

3) End Game

If the player passes all levels successfully, end game menu should be displayed. The player should use ↑ and ↓ keys to navigate between options, and press **SPACE** to confirm the selection. The elapsed time and death count must be displayed on the screen. The screen can be seen in Figure 12.

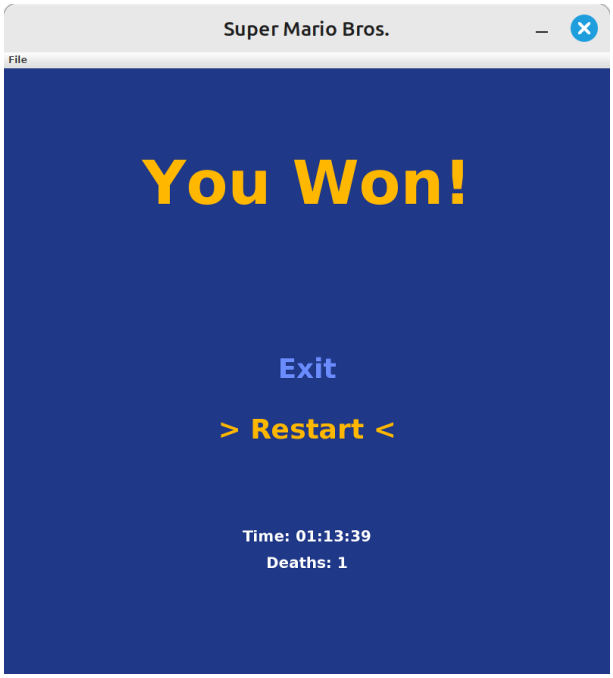


Figure 12: End game menu.

Custom Level

As the final level, you are expected to introduce a new level. You may either add a new enemy type or introduce a new gameplay mechanic.

Level Ideas

- Add a new enemy type with different movement or attack behavior. For example, an enemy may fire a projectile (such as a ball or bullet) that kills the player upon collision.
- A new mechanic is also acceptable, such as edible mushroom-like items that upgrade or downgrade the player, as long as it is more than a simple parameter change to an existing mechanic.

Optional Features

You may also add optional features, such as:

- a new map,
- enemy death animations,
- sound effects.

Assets and Style Notes

- Images for the new enemy type must be your own. You may create them manually, use royalty-free images, or generate them with tools such as ChatGPT or Gemini.
- Keep image sizes reasonably small for submission.
- You may use pixel-art tools to match the retro style of the game.

Data Files

For constructing the map and levels, four `.txt` files are provided.

- `obstacles.txt` provides the coordinates and sizes of the obstacles.
- `pipes.txt` provides the coordinates and sizes of the rectangles that form the start and end pipes.
- `portals.txt` provides the coordinates and sizes of the rectangles that form the red portals in the game.
- `coins.txt` provides the coordinates and sizes of the collectible coins.

The provided coordinates and sizes must be adjusted if you change the canvas size or scaling. Each line follows the format `x,y,halfWidth,halfHeight`. To construct the map, you **must** read these files in `Main.java`. Hardcoding map coordinates directly inside `Map.java` is not allowed.

Project Architecture

The project **must** include the classes listed below. Although the responsibilities of these classes must remain unchanged, you may change the arguments and add or remove fields or methods. Your implementation must include the following classes with clearly defined responsibilities:

- `Main`: Initializes canvas, loads map coordinate data from `data/`, creates the fixed `Map`, builds levels, and starts the game.
- `Mario`: It handles input detection, manages its own state (sprites, facing direction), and checks enemy collisions. Determines movement and draws the Mario. A UML class diagram for the class provided in the Figure 13. It should also handle the respawn and death of the Mario as well.

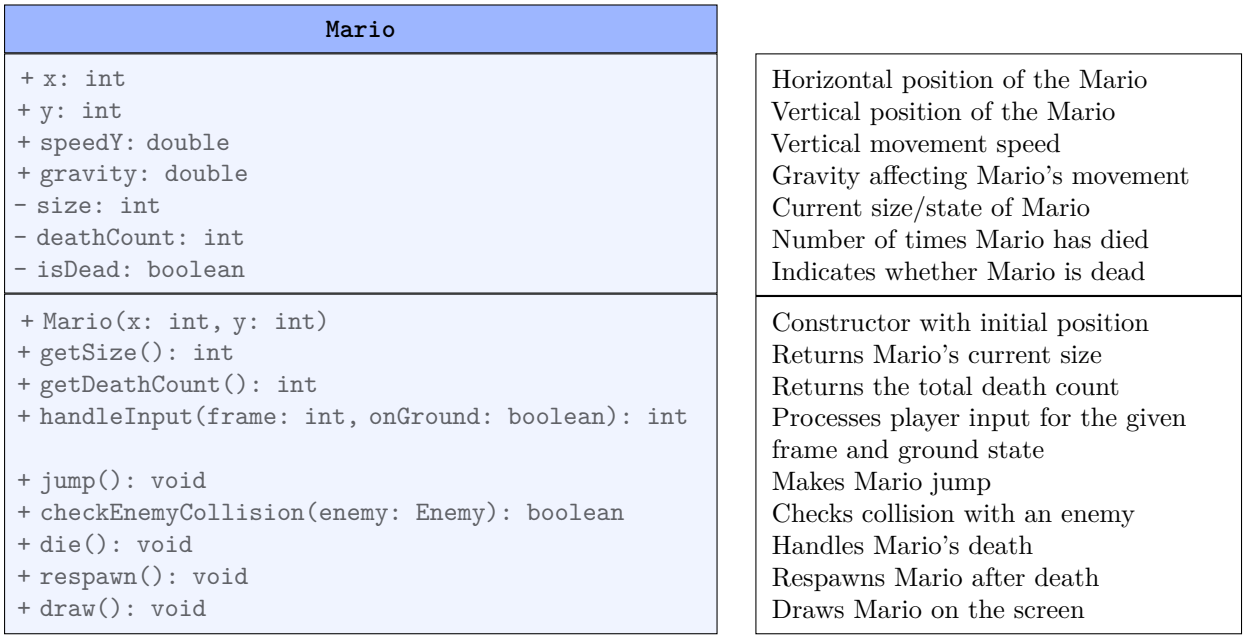


Figure 13: UML Diagram of the `Mario` class.

- `Enemy`: This class is responsible for creating and patrolling the enemy within given coordinates. Manages its own movement and rendering. UML class diagram is given in Figure 14.

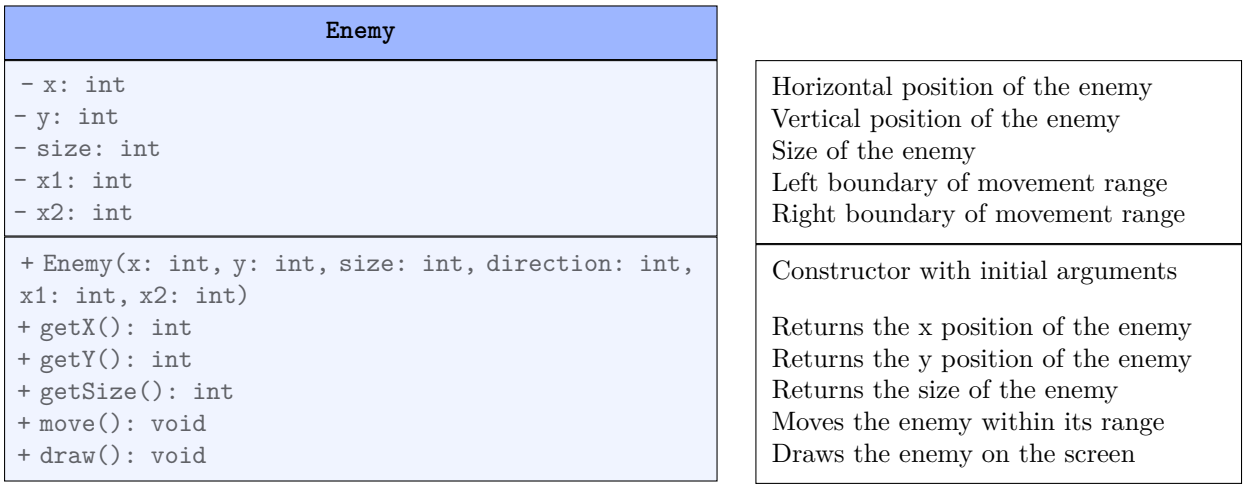


Figure 14: UML Diagram of the `Enemy` class.

- `Level`: This class is responsible for string the information for a level. It stores ID, enemy list, clue text, and coin coordinates. Does not contain fixed world geometry (that belongs to the `Map`). The UML class diagram of the `Level` class is provided in Figure 15.

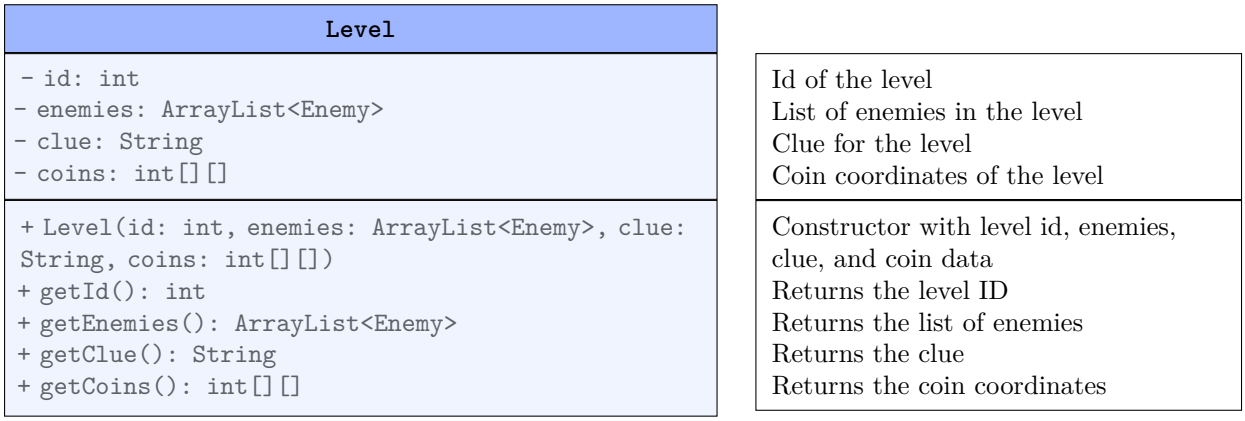


Figure 15: UML Diagram of the `Level` class.

- `Map`: Holds fixed world geometry (obstacles, pipes, portals) shared across all levels. Provides position validation, collision detection, coin collection, portal teleportation, and exit detection if the player reaches to the exit pipe.

Map	
- obstacles: int[][] - pipes: int[][] - portals: int[][] - coins: int[][]	Array of obstacle positions Array of pipes positions Array of portal positions Array of coin positions
+ Map(obstacles: int[][], pipes: int[][], portals: int[][]) + isOnGround(x: int, y: int, halfSize: double, speedY: double, worldW: int, worldH: int): boolean + isOnCeiling(x: int, y: int, halfSize: double, speedY: double, worldW: int, worldH: int): boolean + collectCoin(mario: Mario): int + handleTeleport(mario: Mario, sPressed: boolean): void + isAtExit(mario: Mario): boolean - checkCollision(nextX: double, nextY: double, playerHalfSize: double, obstacles: int[][]): boolean	Constructor with obstacle, pipe, and portal data Checks whether the player is standing on the ground Checks whether the player is touching the ceiling Collects coin touched by Mario Handles portal teleportation for Mario Checks if Mario has reached the exit Checks collision with the given obstacle array

Figure 16: UML Diagram of the `Map` class.

- `Game`: It is the main orchestrator class. This class manages the game loop, connects Mario's input intent to Map's position validation, handles physics coordination, level progression, camera, HUD, and screen transitions. Determines if a level is complete and make level progression. UML class diagram provided in Figure 17.

Game	
- levels: ArrayList<Level> - map: Map - width: int - height: int	List of levels in the game Map used in the game Width of the game window Height of the game window
+ Game(levels: ArrayList<Level>, map: Map) + run(): void - applyGravity(mario: Mario): void - startScreen(mario: Mario): void - levelBanner(id: int, currentLevel: Level, currentEnemies: ArrayList<Enemy>, mario: Mario, elapsedTime: String): long - endScreen(elapsedTime: String, deathCount: int): int - getTime(startingTime: long): String	Constructor with levels and map Runs the main game loop Applies gravity to Mario Displays the start screen Shows the level pass banner and returns timing information Displays the end screen and returns a result code Returns formatted elapsed time from the starting time

Figure 17: UML Diagram of the `Game` class.

Report Format

Your report should be concise and consist of the following sections:

- **Introduction:** Provide an overview of the assignment and the library (`StdDraw`) used in your implementation. The introduction should not exceed half a page.
- **Game Dynamics:** Explain the core mechanics of your implementation, including collision detection, level mechanics, level progression, teleportation, and camera mode, as well as how the classes interact with each other. Provide UML diagrams for each class with explanations. Also include screenshots from different levels of your game.
- **Custom Level:** Explain your custom level by describing any new mechanics and enemy types. Provide screenshots from that level.

Your report should **properly reference all figures, tables, and equations**. Figures and tables should be numbered and cited appropriately (as shown in Figure 1, etc.). Each figure must have a caption placed below it, clearly explaining its content. Each table must have a caption positioned above it. Lastly, all equations should be centered and numbered for reference.

Notes

- You **must** use the **StdDraw graphics library**.
- In all assignments, do not use topics we have not covered in the lectures.
- Make sure you adhere to the submission guidelines and format. If you do not follow the format you will get `-20 points`.
- Make sure your submission can be run without any error. We will run your solution with adding the library to the path like described before in slides. Any error will be punished with `-30 points`. Please make sure your submission can be run with provided file structure.

Submission Files

1. Java source code (**.java file**). Submission details are explained in greater detail in the Submission Guide section.
2. Report (**.pdf format**) explaining your implementation, including screenshots of your gameplay covering different cases. The report must be written using LaTeX. Both the `.pdf` and the `.tex` source files must be submitted. The report should not exceed 10 pages.
3. Provide **a YouTube link at the beginning of your report** showing the gameplay. Ensure that all possible features of the game are visible in the videos.

Submission Guide

Submission Files

Submit a single compressed (`.zip`) file named `NameSurname.zip` to Moodle. The `.zip` file should contain all source code files under the `src` directory, the report in `.pdf` format, and the LaTeX file of the report under `report/` directory. Game assets (images, sounds, etc.) should be placed under `assets`, game data provided in `data` in and any additional optional files (if needed) should be placed under `misc`. Name the main Java file `Main.java`. Name your report `NameSurname.pdf` and LaTeX file as `NameSurname.tex`. **Do not use Turkish characters** in filenames, code, or comments. At the top of each `.java` file, include your name and student number as a comment.

```
NameSurname.zip
|-- src
|   |-- Main.java
|   |-- Game.java
|   |-- Map.java
|   |-- Player.java
|   |-- Enemy.java
|   `-- Level.java
|-- report
|   |-- NameSurname.pdf
|   `-- NameSurname.tex
|-- assets
|   `-- (all required images)
|-- data
|   |-- obstacles.txt
|   |-- pipes.txt
|   |-- portals.txt
|   `-- coins.txt
`-- misc
    `-- (optional files)
```

Late Submission Policy

Maximum late duration is two days. Late submissions are graded at 50%.

Mandatory Submission

Assignment submission is mandatory.

AI Usage Policy

Coding

- You may use AI tools for coding assistance.
- You must understand and explain every submitted line.
- Fully AI-generated submissions are not accepted.

Report

- The report content must be written by you.
- AI tools may only be used for grammar/language corrections.

Plagiarism

Plagiarism results in disciplinary action according to university regulations.